Data Structures and Algorithms Lab
Assignment 2
Experiment 2: To experiment with the concept of pointers, structure, and dynamic memory allocation to realize linked lists, their types, and application.

Instruction: Make a pdf file of the program code in C and its output.

Name: Aditi Srivastava
SAP ID: 590026127
Batch: 39
Link of your GitHub Account: https://github.com/aditisrivastava12729-hub
Course: B.Tech. CSE (Sem II)
Subject: DSA Lab
Date: 17 February 2026
Note: Do all the implementations in C programming.

1. Write a C program to traverse a singly linked list and count the number of nodes present in the list.

```c
#include <stdio.h>
#include <stdlib.h>

struct node {
    int data;
    struct node* next;
};

int main() {
    struct node *head = NULL, *temp, *newnode;
    int n, i, count = 0;

    printf("Enter number of nodes: ");
    scanf("%d", &n);

    for(i = 0; i < n; i++) {
        newnode = (struct node*)malloc(sizeof(struct node));
        printf("Enter data: ");
        scanf("%d", &newnode->data);
        newnode->next = NULL;

        if(head == NULL)
            head = temp = newnode;
        else {
            temp->next = newnode;
            temp = newnode;
```

```c
        }
    }

    temp = head;
    while(temp != NULL) {
        printf("%d -> ", temp->data);
        count++;
        temp = temp->next;
    }

    printf("NULL\nTotal Nodes = %d", count);
    return 0;
}
```

```
Output

Enter number of nodes: 3
Enter data: 10
Enter data: 20
Enter data: 30
10 -> 20 -> 30 -> NULL
Total Nodes = 3

=== Code Execution Successful ===
```

2. Write a C program to implement a singly linked list and search for a given element in the list. If the element is found, display the position of the element in the linked list; otherwise, display element does not exit.

```c
#include <stdio.h>
#include <stdlib.h>

struct node {
    int data;
    struct node* next;
};

int main() {
    struct node *head = NULL, *temp, *newnode;
    int n, i, key, pos = 1, found = 0;

    printf("Enter number of nodes: ");
    scanf("%d", &n);

    for(i = 0; i < n; i++) {
        newnode = (struct node*)malloc(sizeof(struct node));
        scanf("%d", &newnode->data);
        newnode->next = NULL;

        if(head == NULL)
            head = temp = newnode;
        else {
            temp->next = newnode;
            temp = newnode;
        }
```

```
Output

Enter number of nodes: 3
10 20 30
Enter element to search: 30
Element found at position 3

=== Code Execution Successful ===
```

3. Write a C program to implement insertion operations in a singly linked list. The program should perform the following operations:
    - Insert a new element at the beginning of the linked list.
    - Insert a new element at the end of the linked list.
    - Insert a new element after a given node in the linked list.

```c
#include <stdio.h>
#include <stdlib.h>

struct node {
    int data;
    struct node* next;
};

void display(struct node* head) {
    while(head != NULL) {
        printf("%d -> ", head->data);
        head = head->next;
    }
    printf("NULL\n");
}

int main() {
    struct node *head = NULL, *temp, *newnode;
    int choice, val, key;

    while(1) {
        printf("\n1.Insert Beginning\n2.Insert End\n3.Insert After\n4.Display\n5.Exit\n");
        scanf("%d", &choice);

        if(choice == 5) break;
```

```c
switch(choice) {
    case 1:
        printf("Enter value: ");
        scanf("%d", &val);
        newnode = malloc(sizeof(struct node));
        newnode->data = val;
        newnode->next = head;
        head = newnode;
        break;

    case 2:
        printf("Enter value: ");
        scanf("%d", &val);
        newnode = malloc(sizeof(struct node));
        newnode->data = val;
        newnode->next = NULL;

        if(head == NULL)
            head = newnode;
        else {
            temp = head;
            while(temp->next != NULL)
                temp = temp->next;
            temp->next = newnode;
        }
        break;
```

```c
                break;

            case 3:
                printf("Enter key and value: ");
                scanf("%d%d", &key, &val);
                temp = head;
                while(temp != NULL && temp->data != key)
                    temp = temp->next;

                if(temp != NULL) {
                    newnode = malloc(sizeof(struct node));
                    newnode->data = val;
                    newnode->next = temp->next;
                    temp->next = newnode;
                }
                break;

            case 4:
                display(head);
                break;
        }
    }
    return 0;
}
```

Output

```
1.Insert Beginning
2.Insert End
3.Insert After
4.Display
5.Exit
10

1.Insert Beginning
2.Insert End
3.Insert After
4.Display
5.Exit
5


=== Code Execution Successful ===
```

4. Write a C program to implement deletion operations on a singly linked list. The program should support deleting an element from:
    - The beginning of the linked list
    - The end of the linked list
    - After a g

```c
#include <stdio.h>
#include <stdlib.h>

struct node {
    int data;
    struct node* next;
};

void display(struct node* head) {
    while(head != NULL) {
        printf("%d -> ", head->data);
        head = head->next;
    }
    printf("NULL\n");
}

int main() {
    struct node *head = NULL, *temp, *newnode, *prev;
    int n, i, choice, key;

    printf("Enter number of nodes: ");
    scanf("%d", &n);

    for(i = 0; i < n; i++) {
        newnode = malloc(sizeof(struct node));
        scanf("%d", &newnode->data);
```

iven node (specified by its value)

```
    newnode->next = NULL;

    if(head == NULL)
        head = temp = newnode
    else {
        temp->next = newnode;
        temp = newnode;
    }
}

printf("1.Delete Beginning\n2
scanf("%d", &choice);

if(choice == 1) {
    temp = head;
    head = head->next;
    free(temp);
}
else if(choice == 2) {
    temp = head;
    while(temp->next->next !=
        temp = temp->next;
    free(temp->next);
    temp->next = NULL;
}
else if(choice == 3) {
```

```c
        free(temp);
    }
    else if(choice == 2) {
        temp = head;
        while(temp->next->next != NULL)
            temp = temp->next;
        free(temp->next);
        temp->next = NULL;
    }
    else if(choice == 3) {
        printf("Enter value after which delete: ");
        scanf("%d", &key);
        temp = head;
        while(temp->data != key)
            temp = temp->next;

        prev = temp->next;
        temp->next = prev->next;
        free(prev);
    }

    printf("Updated List: ");
    display(head);

    return 0;
}
```

Output

```
Enter number of nodes: 5
1 2 3 4 5
1.Delete Beginning
2.Delete End
3.Delete After
Enter choice: 2
Updated List: 1 -> 2 -> 3 -> 4 -> NULL


=== Code Execution Successful ===
```

5. Write a C program to reverse the data of the nodes of a singly linked list without changing the links between the nodes.

```c
#include <stdio.h>
#include <stdlib.h>

struct node {
    int data;
    struct node* next;
};

void display(struct node* head) {
    while(head != NULL) {
        printf("%d -> ", head->data);
        head = head->next;
    }
    printf("NULL\n");
}

int main() {
    struct node *head = NULL, *temp, *newnode, *i, *j;
    int n, k, swap;

    printf("Enter number of nodes: ");
    scanf("%d", &n);

    for(k = 0; k < n; k++) {
        newnode = malloc(sizeof(struct node));
        scanf("%d", &newnode->data);
```

```c
        scanf("%d", &newnode->data);
        newnode->next = NULL;

        if(head == NULL)
            head = temp = newnode;
        else {
            temp->next = newnode;
            temp = newnode;
        }
    }

    for(i = head; i != NULL; i = i->next) {
        for(j = i->next; j != NULL; j = j->next) {
            if(i->data > j->data) {
                swap = i->data;
                i->data = j->data;
                j->data = swap;
            }
        }
    }

    printf("Reversed Data List: ");
    display(head);

    return 0;
}
```

Output

```
Enter number of nodes: 3
30 20 10
Reversed Data List: 10 -> 20 -> 30 -> NULL


=== Code Execution Successful ===
```

6. Write a C program to implement a Circular Singly Linked List and perform all basic operations on it. The program must support the following operations:
    - Creation of a Circular Linked List
    - Insertion of a node (At the beginning and at the end of the list)
    - Deletion of a node (From the beginning and from the end of the list)
    - Traversal and display of the circular linked list and counting the number of nodes in the list.

```c
#include <stdio.h>
#include <stdlib.h>

struct node {
    int data;
    struct node* next;
};

int main() {
    struct node *head = NULL, *temp, *newnode;
    int n, i, count = 0;

    printf("Enter number of nodes: ");
    scanf("%d", &n);

    for(i = 0; i < n; i++) {
        newnode = malloc(sizeof(struct node));
        scanf("%d", &newnode->data);

        if(head == NULL) {
            head = temp = newnode;
            newnode->next = head;
        } else {
            temp->next = newnode;
            newnode->next = head;
            temp = newnode;
```

```c
        }
    }

    temp = head;
    printf("Circular List: ");
    do {
        printf("%d -> ", temp->data);
        count++;
        temp = temp->next;
    } while(temp != head);

    printf("(back to head)");
    printf("\nTotal Nodes = %d", count);

    return 0;
}
```

```
Output
Enter number of nodes: 3
4 5 6
Circular List: 4 -> 5 -> 6 -> (back to head)
Total Nodes = 3

=== Code Execution Successful ===
```

7. Write a C program to implement a Doubly Linked List. The program should perform all basic operations on the list. The program must support the following operations:
    - Creation of a Doubly Linked List, display and counting the number of nodes
    - Insertion of a node (At the beginning, At the end, After a given node)
    - Deletion of a node (From the beginning, From the end, of a given node) □ Traversal of the list (Forward traversal and Backward traversal)

```c
#include <stdio.h>
#include <stdlib.h>

struct node {
    int data;
    struct node *prev, *next;
};

int main() {
    struct node *head = NULL, *temp, *newnode;
    int n, i, count = 0;

    printf("Enter number of nodes: ");
    scanf("%d", &n);

    for(i = 0; i < n; i++) {
        newnode = malloc(sizeof(struct node));
        scanf("%d", &newnode->data);
        newnode->next = NULL;
        newnode->prev = temp;

        if(head == NULL)
            head = newnode;
        else
            temp->next = newnode;
```

```c
            head = newnode;
        else
            temp->next = newnode;

        temp = newnode;
    }

    temp = head;
    printf("Forward: ");
    while(temp != NULL) {
        printf("%d <-> ", temp->data);
        count++;
        if(temp->next == NULL) break;
        temp = temp->next;
    }

    printf("NULL\nBackward: ");
    while(temp != NULL) {
        printf("%d <-> ", temp->data);
        temp = temp->prev;
    }

    printf("NULL\nTotal Nodes = %d", count);

    return 0;
}
```

Output

```
Enter number of nodes: 3
5 6 7
Forward: 5 <-> 6 <-> 7 <-> NULL
Backward: 7 <-> 6 <-> 5 <-> NULL
Total Nodes = 3

=== Code Execution Successful ===
```

8. Write a C program to implement a Doubly Circular Linked List. The program should perform all basic operations on the list. The program must support the following operations:
    - Creation of a Doubly Circular Linked List, display and counting the number of nodes
    - Insertion of a node (At the beginning, At the end)
    - Deletion of a node (From the beginning, From the end)

```c
#include <stdio.h>
#include <stdlib.h>

struct node {
    int data;
    struct node *prev, *next;
};

int main() {
    struct node *head = NULL, *temp, *newnode;
    int n, i;

    printf("Enter number of nodes: ");
    scanf("%d", &n);

    for(i = 0; i < n; i++) {
        newnode = malloc(sizeof(struct node));
        scanf("%d", &newnode->data);

        if(head == NULL) {
            head = temp = newnode;
            newnode->next = newnode;
            newnode->prev = newnode;
        } else {
            newnode->next = head;
            newnode->prev = temp;
```

```c
        scanf("%d", &newnode->data);

        if(head == NULL) {
            head = temp = newnode;
            newnode->next = newnode;
            newnode->prev = newnode;
        } else {
            newnode->next = head;
            newnode->prev = temp;
            temp->next = newnode;
            head->prev = newnode;
            temp = newnode;
        }
    }

    temp = head;
    printf("Doubly Circular List: ");
    do {
        printf("%d <-> ", temp->data);
        temp = temp->next;
    } while(temp != head);

    printf("(back to head)");

    return 0;
}
```

Output

```
Enter number of nodes: 5
1 2 3 4 5
Doubly Circular List: 1 <-> 2 <-> 3 <-> 4 <-> 5 <-> (back to head)

=== Code Execution Successful ===
```

9. Write a C program to represent and evaluate a Polynomial using a Linked List. Each node should represent a term containing coefficient and exponent.

```c
#include <stdio.h>
#include <stdlib.h>

struct node {
    int coeff, exp;
    struct node* next;
};

int main() {
    struct node *head = NULL, *temp, *newnode;
    int n, i;

    printf("Enter number of terms: ");
    scanf("%d", &n);

    for(i = 0; i < n; i++) {
        newnode = malloc(sizeof(struct node));
        printf("Enter coefficient and exponent: ");
        scanf("%d%d", &newnode->coeff, &newnode->exp);
        newnode->next = NULL;

        if(head == NULL)
            head = temp = newnode;
        else {
            temp->next = newnode;
            temp = newnode;
        }
    }

    temp = head;
    printf("Polynomial: ");
    while(temp != NULL) {
        printf("%dx^%d", temp->coeff, temp->exp);
        if(temp->next != NULL)
            printf(" + ");
        temp = temp->next;
    }

    return 0;
}
```

Output

```
Enter number of terms: 4
Enter coefficient and exponent: 5 7
Enter coefficient and exponent: 5 6
Enter coefficient and exponent: 4 5
Enter coefficient and exponent: 4 5
Polynomial: 5x^7 + 5x^6 + 4x^5 + 4x^5

=== Code Execution Successful ===
```

10. Write a C program to maintain Student Records using a Linked List. Each node should store student details such as roll number, name, and marks. Implement operations to insert, delete, search, and display records.

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

// Define structure (Node)
struct Student {
    int roll;
    char name[50];
    float marks;
    struct Student *next;
};

// Head pointer
struct Student *head = NULL;

// Function to Insert at End
void insert() {
    struct Student *newNode, *temp;
    newNode = (struct Student*)malloc(sizeof(struct Student));

    printf("Enter Roll Number: ");
```

```c
    scanf("%d", &newNode->roll);

    printf("Enter Name: ");
    scanf(" %[^\n]", newNode->name);

    printf("Enter Marks: ");
    scanf("%f", &newNode->marks);

    newNode->next = NULL;

    if (head == NULL) {
        head = newNode;
    } else {
        temp = head;
        while (temp->next != NULL) {
            temp = temp->next;
        }
        temp->next = newNode;
    }

    printf("Record Inserted Successfully!\n");
```

```c
    printf("Record Inserted Successfully!\n");
}

// Function to Delete by Roll Number
void deleteRecord() {
    int roll;
    struct Student *temp = head, *prev = NULL;

    printf("Enter Roll Number to Delete: ");
    scanf("%d", &roll);

    if (head == NULL) {
        printf("List is Empty!\n");
        return;
    }

    // If first node is to be deleted
    if (temp != NULL && temp->roll == roll) {
        head = temp->next;
        free(temp);
        printf("Record Deleted Successfully!\n");
```

```c
        return;
    }

    while (temp != NULL && temp->roll != roll) {
        prev = temp;
        temp = temp->next;
    }

    if (temp == NULL) {
        printf("Record Not Found!\n");
        return;
    }

    prev->next = temp->next;
    free(temp);
    printf("Record Deleted Successfully!\n");
}

// Function to Search Record
void search() {
    int roll;
```

```c
    printf("Enter Roll Number to Search: ");
    scanf("%d", &roll);

    while (temp != NULL) {
        if (temp->roll == roll) {
            printf("\nRecord Found!\n");
            printf("Roll: %d\nName: %s\nMarks: %.2f\n",
                   temp->roll, temp->name, temp->marks);
            return;
        }
        temp = temp->next;
    }

    printf("Record Not Found!\n");
}

// Function to Display All Records
void display() {
    struct Student *temp = head;

    if (head == NULL) {
        printf("No Records Available!\n");
```

```c
        return;
    }

    printf("\nStudent Records:\n");
    while (temp != NULL) {
        printf("Roll: %d | Name: %s | Marks: %.2f\n",
                temp->roll, temp->name, temp->marks);
        temp = temp->next;
    }
}

// Main Function
int main() {
    int choice;

    while (1) {
        printf("\n--- Student Record Management (Linked List)
            ---\n");
        printf("1. Insert Record\n");
        printf("2. Delete Record\n");
        printf("3. Search Record\n");
        printf("4. Display Records\n");
```

```c
        ---\n");
    printf("1. Insert Record\n");
    printf("2. Delete Record\n");
    printf("3. Search Record\n");
    printf("4. Display Records\n");
    printf("5. Exit\n");
    printf("Enter your choice: ");
    scanf("%d", &choice);

    switch (choice) {
        case 1: insert(); break;
        case 2: deleteRecord(); break;
        case 3: search(); break;
        case 4: display(); break;
        case 5: exit(0);
        default: printf("Invalid Choice!\n");
    }
}

return 0;
```

```
--- Student Record Management (Linked List) ---
1. Insert Record
2. Delete Record
3. Search Record
4. Display Records
5. Exit
Enter your choice: 1
Enter Roll Number: 24
Enter Name: aditi
Enter Marks: 90
Record Inserted Successfully!
```

Thank you